Application Integration

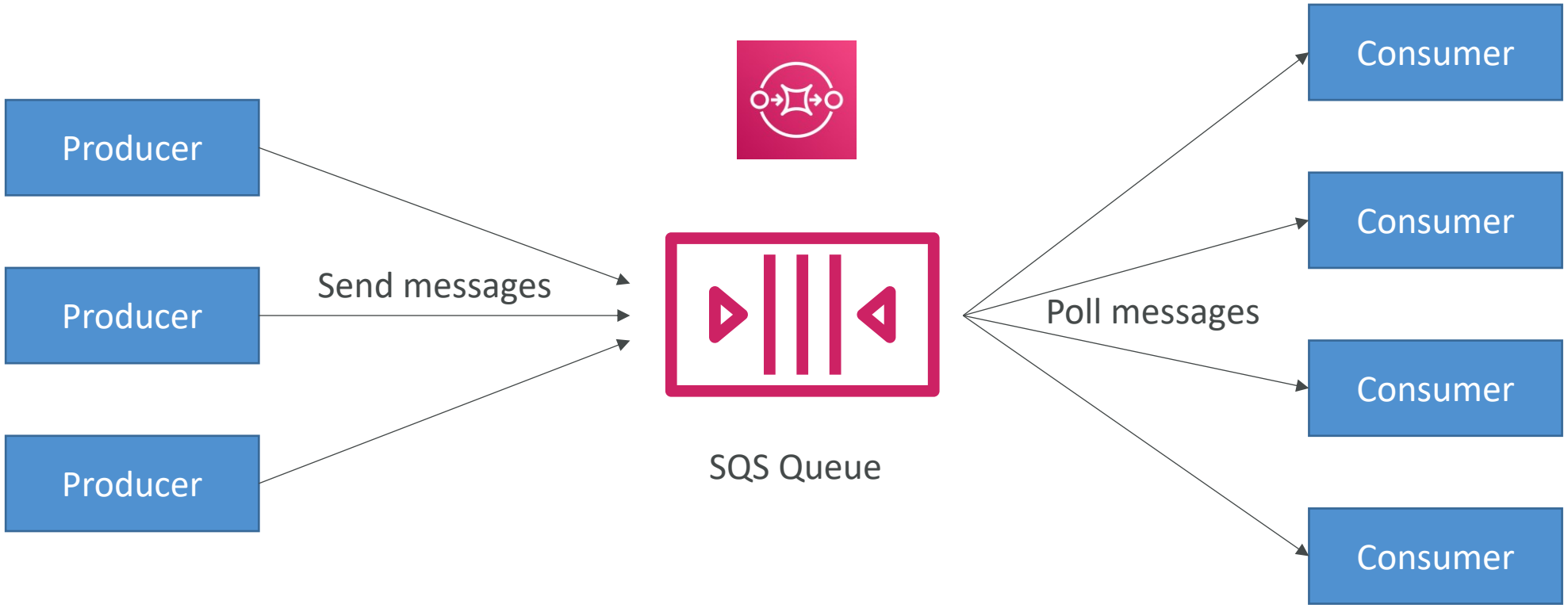
Moving data between systems in AWS

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

DataCumulus Sundog Education

AWS SQS
What's a queue?



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

AWS SQS – Standard Queue

- Oldest offering (over 10 years old)
- Fully managed
- Scales from 1 message per second to 15,000s per second
- Default retention of messages: 4 days, maximum of 14 days
- No limit to how many messages can be in the queue
- Low latency (<10 ms on publish and receive)
- Horizontal scaling in terms of number of consumers
- Can have duplicate messages (at least once delivery, occasionally)
- Can have out of order messages (best effort ordering)
- Limitation of 256KB per message sent

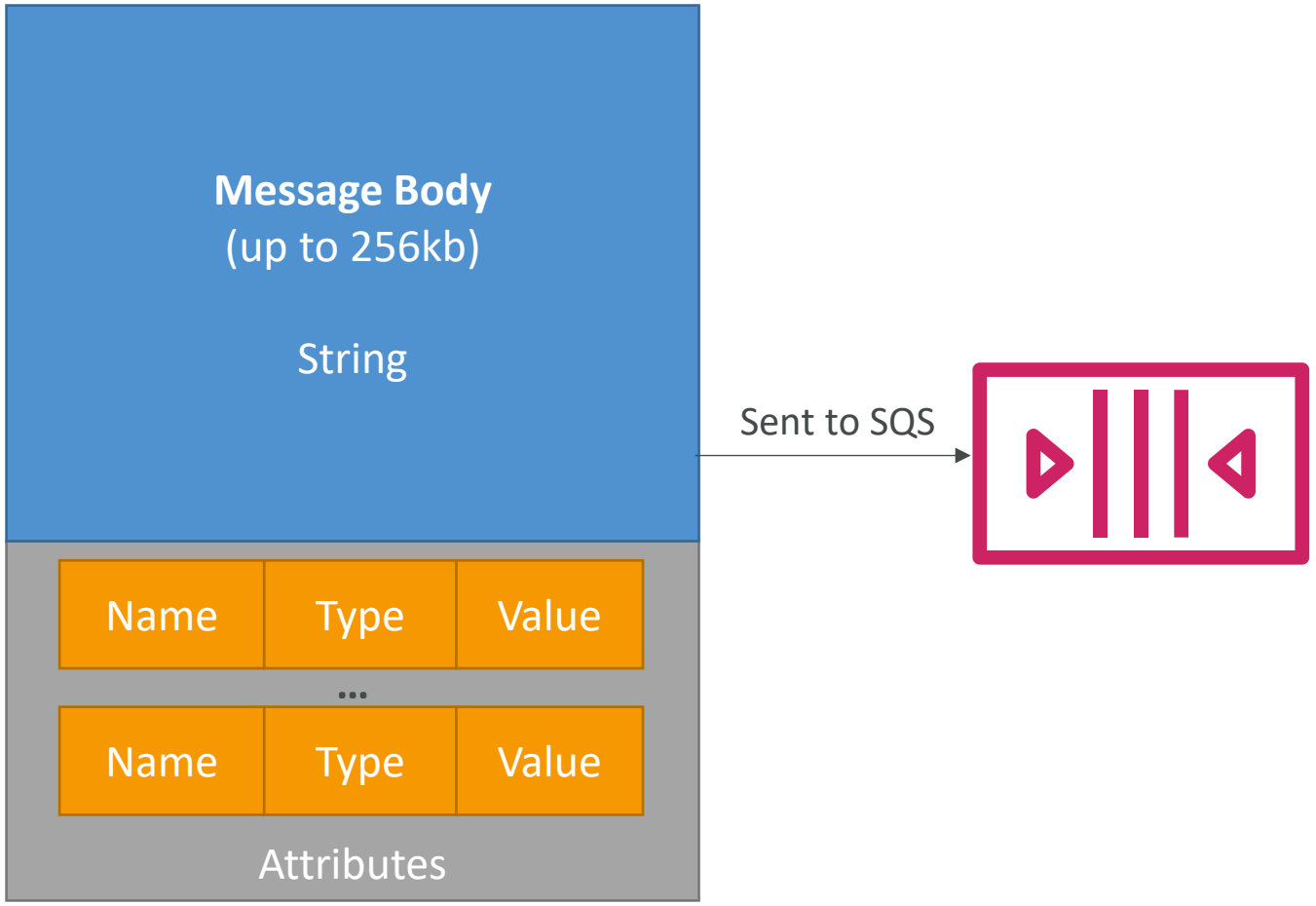
NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

SQS – Producing Messages

- Define Body
- Add message attributes (metadata – optional)
- Provide Delay Delivery (optional)

- Get back
  - Message identifier
  - MD5 hash of the body

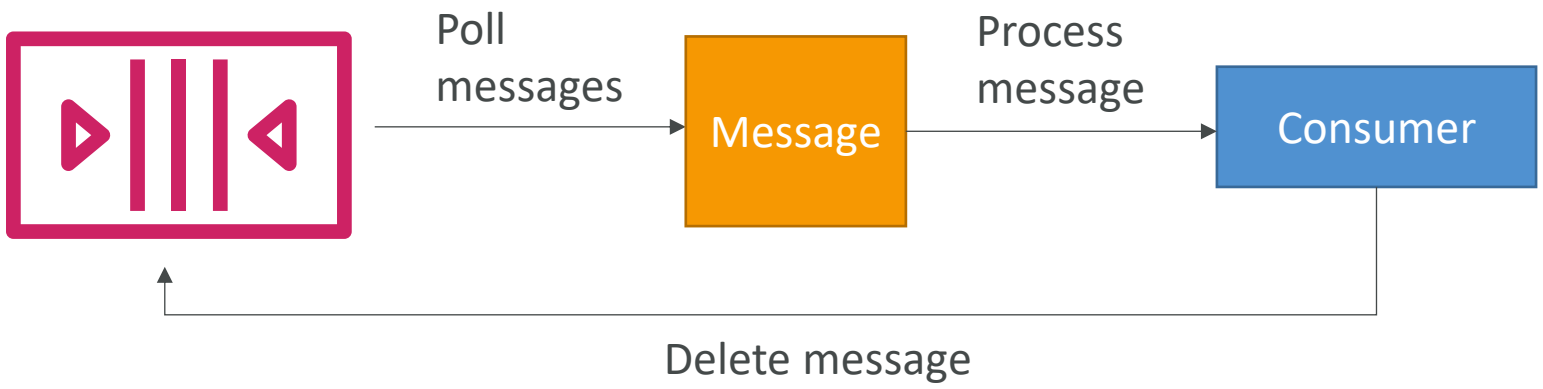


NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

SQS – Consuming Messages

- Consumers…
- Poll SQS for messages (receive up to 10 messages at a time)
- Process the message within the visibility timeout
- Delete the message using the message ID & receipt handle

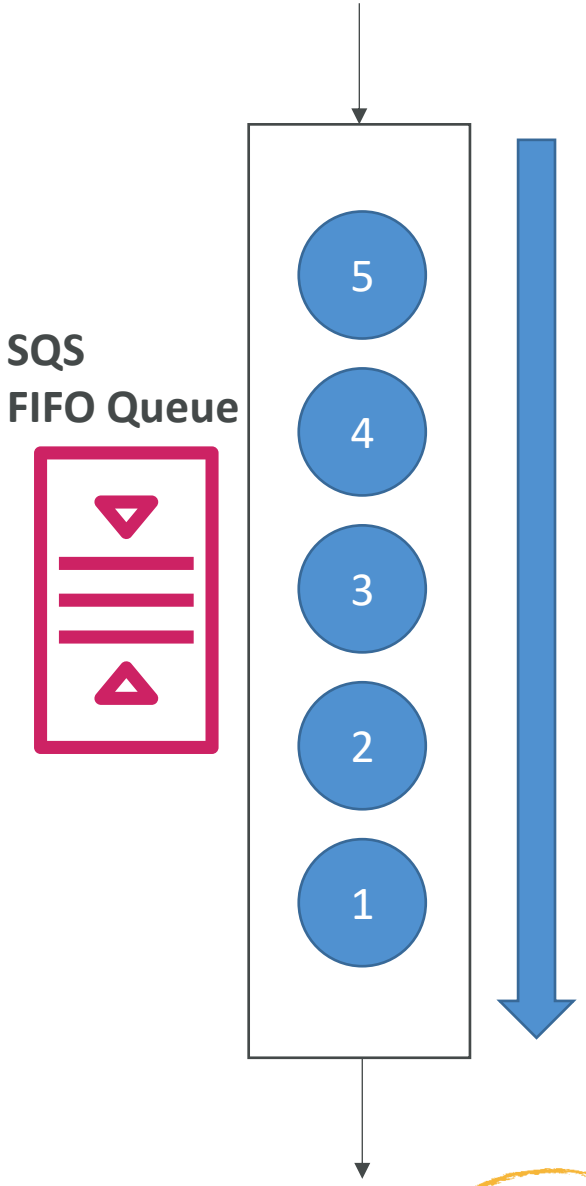


© 2023 All Rights Reserved Worldwide

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

AWS SQS – FIFO Queue

- Newer offering (First In - First out) – not available in all regions!
- Name of the queue must end in .fifo
- Lower throughput (up to 3,000 per second with batching, 300/s without)
- Messages are processed in order by the consumer
- Messages are sent exactly once
- 5-minute interval de-duplication using "Duplication ID"

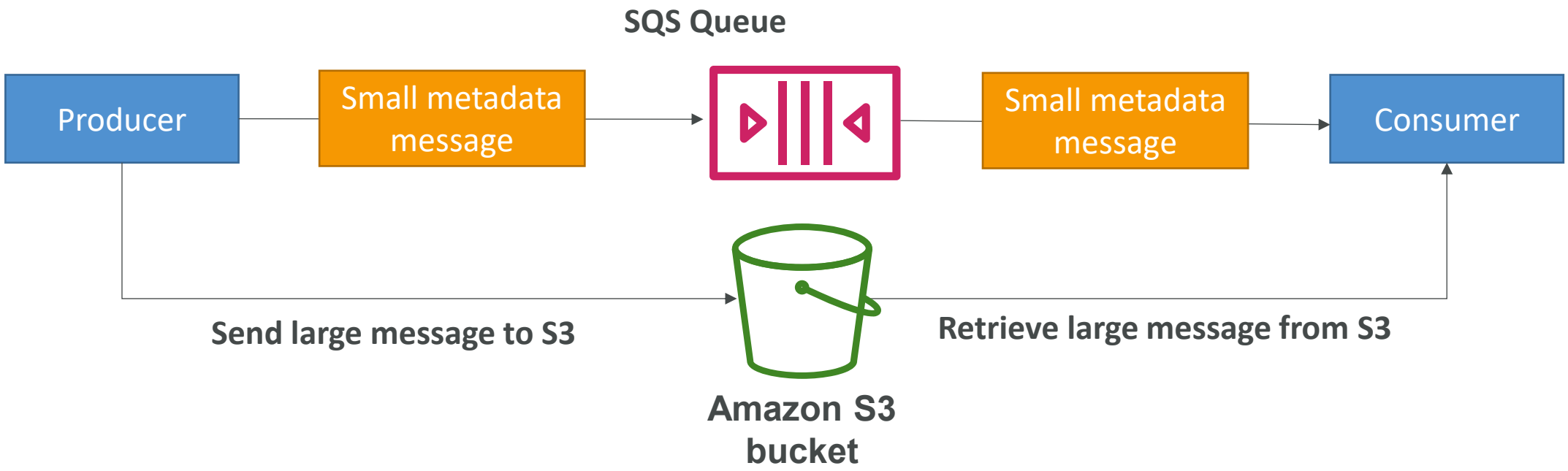


NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

SQS Extended Client

- Message size limit is 256KB, how to send large messages?
- Using the SQS Extended Client (Java Library)



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

AWS SQS Use Cases

- Decouple applications
  (for example to handle payments asynchronously)
- Buffer writes to a database
  (for example a voting application)
- Handle large loads of messages coming in
  (for example an email sender)

- SQS can be integrated with Auto Scaling through CloudWatch!

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

SQS Limits

- Maximum of 120,000 in-flight messages being processed by consumers
- Batch Request has a maximum of 10 messages – max 256KB
- Message content is XML, JSON, Unformatted text
- Standard queues have an unlimited TPS
- FIFO queues support up to 3,000 messages per second (using batching)
- Max message size is 256KB (or use Extended Client)
- Data retention from 1 minute to 14 days
- Pricing:
  - Pay per API Request
  - Pay per network usage

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

AWS SQS Security

- Encryption in flight using the HTTPS endpoint
- Can enable SSE (Server Side Encryption) using KMS
  - Can set the CMK (Customer Master Key) we want to use
  - SSE only encrypts the body, not the metadata (message ID, timestamp, attributes)
- IAM policy must allow usage of SQS
- SQS queue access policy
  - Finer grained control over IP
  - Control over the time the requests come in

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Kinesis Data Stream vs SQS

- **Kinesis Data Stream:**
  - Data can be consumed many times
  - Data is deleted after the retention period
  - Ordering of records is preserved (at the shard level) – even during replays
  - Build multiple applications reading from the same stream independently (Pub/Sub)
  - “Streaming MapReduce” querying capability (Spark, Flink…)
  - Checkpointing needed to track progress of consumption (ex: KCL with DynamoDB)
  - Provisioned mode or on-demand mode
- **SQS:**
  - Queue, decouple applications
  - One application per queue
  - Records are deleted after consumption (ack / fail)
  - Messages are processed independently for standard queue
  - Ordering for FIFO queues (decreased throughtput)
  - Capability to “delay” messages
  - Dynamic scaling of load (no-ops)

© 2023 All Rights Reserved Worldwide
NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Kinesis Data Streams vs SQS

	Kinesis Data Streams	Kinesis Data Firehose	Amazon SQS Standard	Amazon SQS FIFO
Managed by AWS	yes	yes	yes	yes
Ordering	Shard / Key	No	No	Specify Group ID
Delivery	At least once	At least once	At least once	Exactly Once
Replay	Yes	No	No	No
Max Data Retention	365 days	No	14 days	14 days
Scaling	Provision Shards: 1MB/s producer 2MB/s consumer On-demand mode	No limit	No limit	With batching: - 3,000 msg/s - 30,000 msg/s in high throughput mode
Max Object Size	1MB	128 MB at destination	256KB (more if using extended lib)	256KB (more if using extended lib)

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

SQS vs Kinesis – Use cases

- **SQS use cases:**
  - Order processing
  - Image Processing
  - Auto scaling queues according to messages.
  - Buffer and Batch messages for future processing.
  - Request Offloading

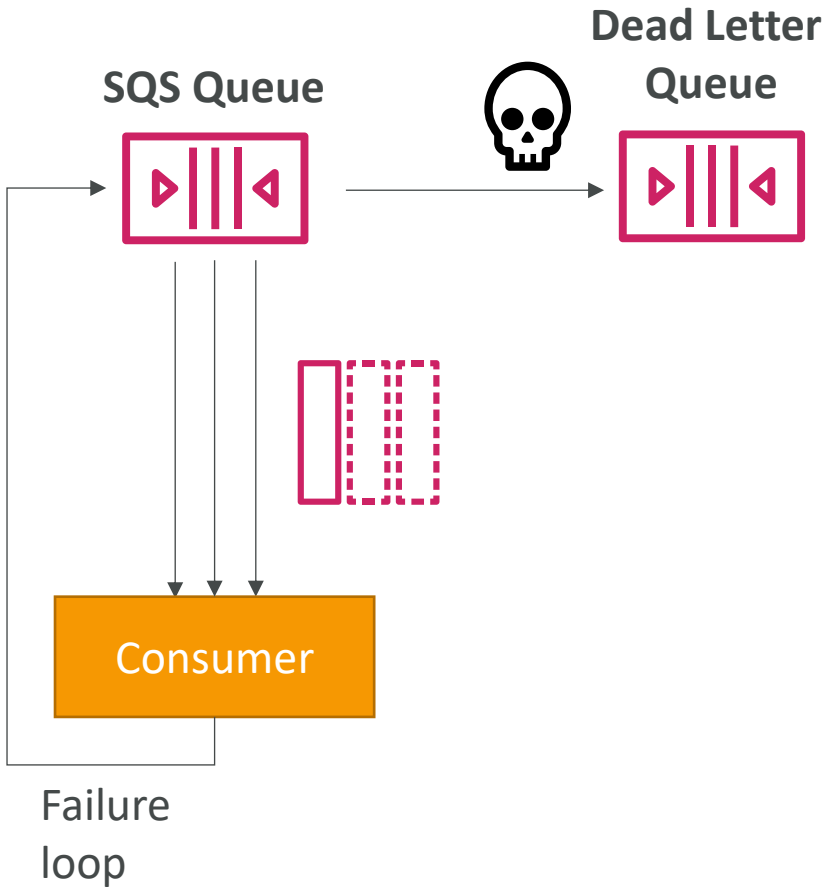
- **Kinesis Data Streams use cases:**
  - Fast log and event data collection and processing
  - Real Time metrics and reports
  - Mobile data capture
  - Real Time data analytics
  - Gaming data feed
  - Complex Stream Processing
  - Data Feed from “Internet of Things”

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Amazon SQS – Dead Letter Queue (DLQ)

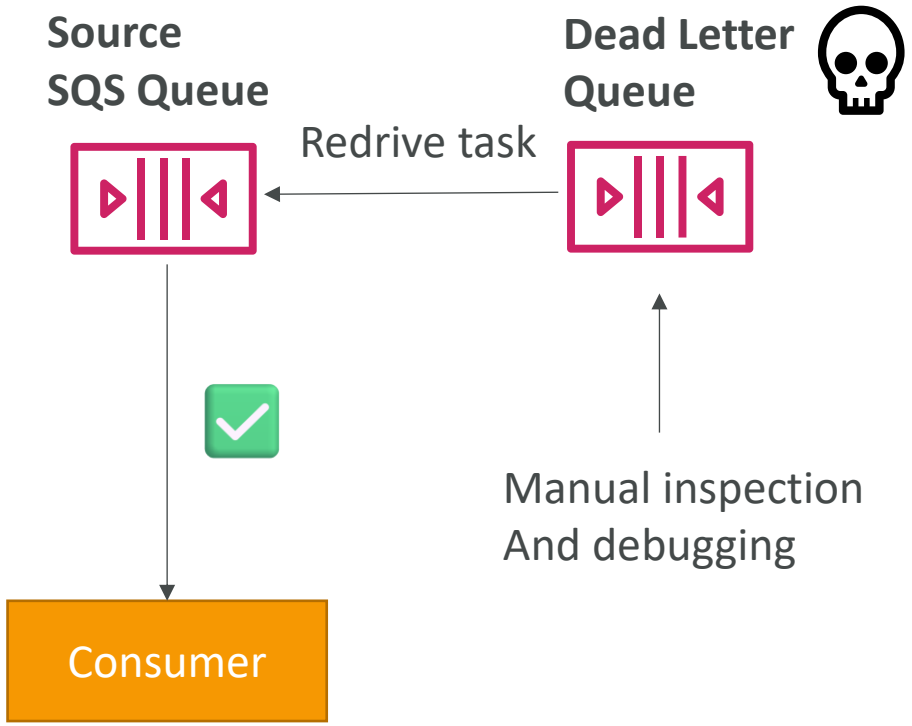
- If a consumer fails to process a message within the Visibility Timeout…
  the message goes back to the queue!
- We can set a threshold of how many times a message can go back to the queue
- After the **MaximumReceives** threshold is exceeded, the message goes into a Dead Letter Queue (DLQ)
- Useful for debugging!
- **DLQ of a FIFO queue must also be a FIFO queue**
- **DLQ of a Standard queue must also be a Standard queue**
- Make sure to process the messages in the DLQ before they expire:
  - Good to set a retention of 14 days in the DLQ



© 2023 All Rights Reserved Worldwide
NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

SQS DLQ – Redrive to Source

- Feature to help consume messages in the DLQ to understand what is wrong with them
- When our code is fixed, we can redrive the messages from the DLQ back into the source queue (or any other queue) in batches without writing custom code

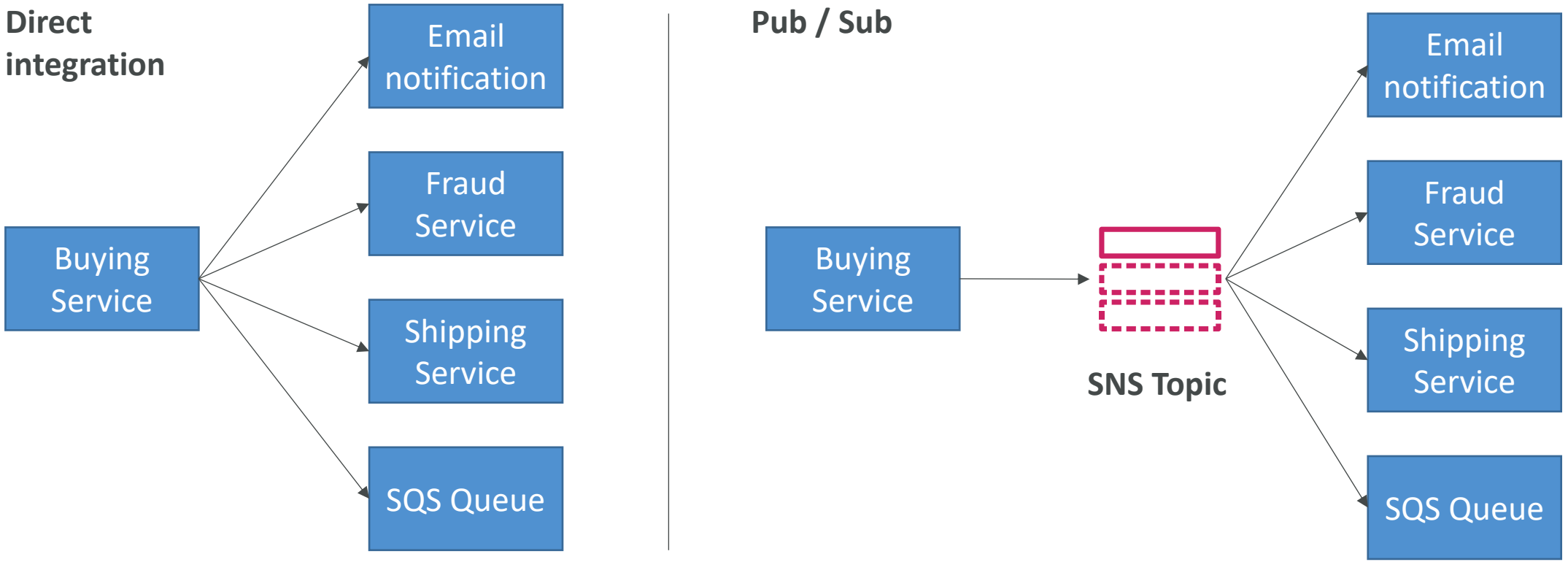


NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Amazon SNS

- What if you want to send one message to many receivers?

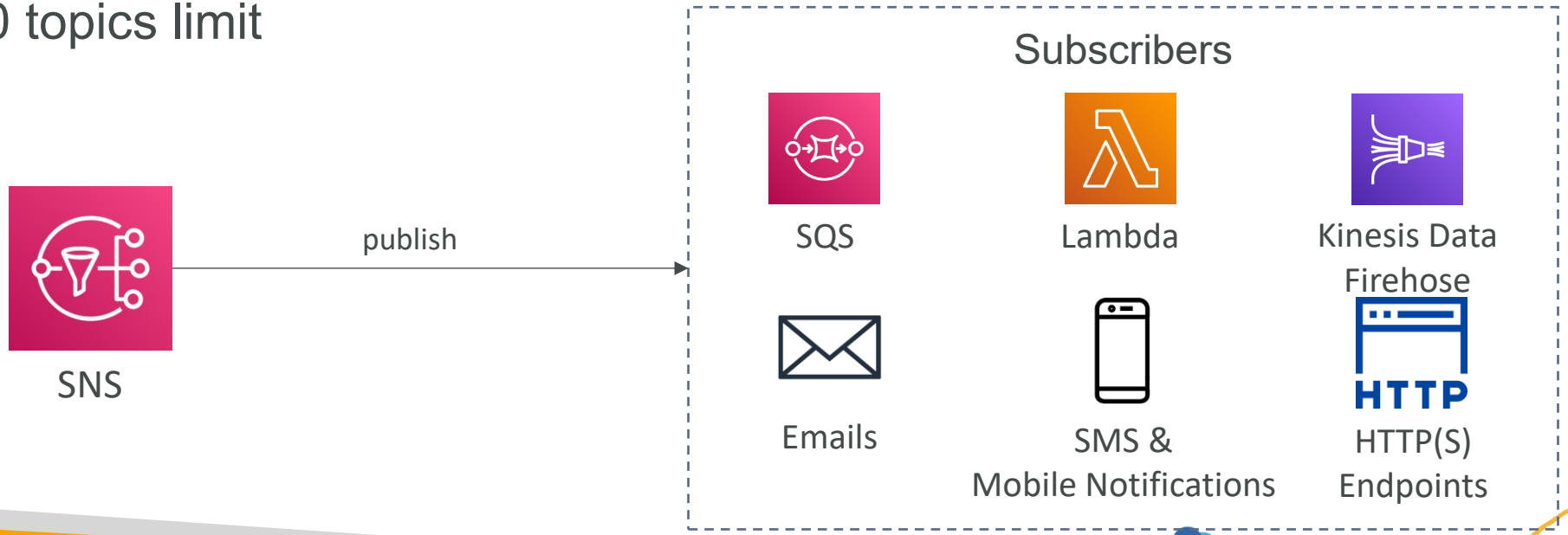


NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Amazon SNS

- The “event producer” only sends message to one SNS topic
- As many “event receivers” (subscriptions) as we want to listen to the SNS topic notifications
- Each subscriber to the topic will get all the messages (note: new feature to filter messages)
- Up to 12,500,000 subscriptions per topic
- 100,000 topics limit

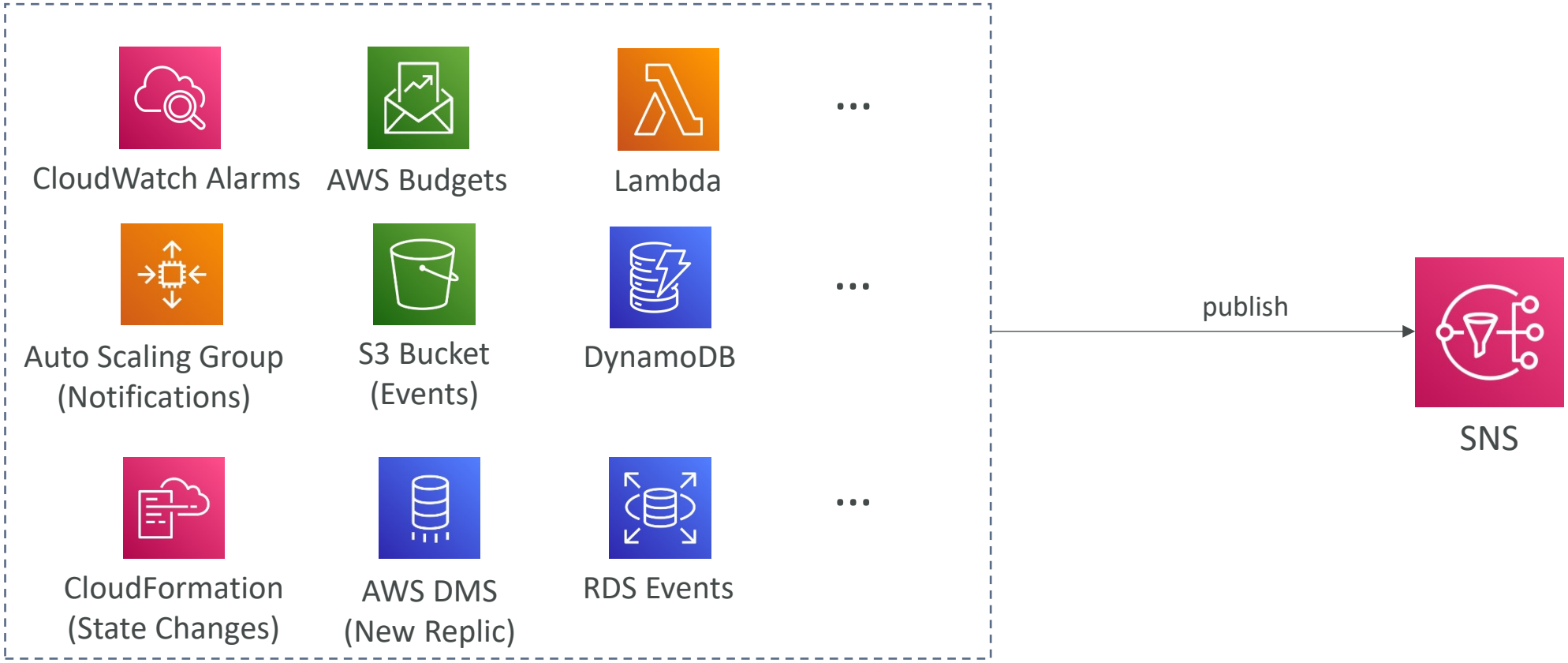


NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

SNS integrates with a lot of AWS services

- Many AWS services can send data directly to SNS for notifications



© 2023 All Rights Reserved Worldwide

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon SNS – How to publish

- Topic Publish (using the SDK)
  - Create a topic
  - Create a subscription (or many)
  - Publish to the topic

- Direct Publish (for mobile apps SDK)
  - Create a platform application
  - Create a platform endpoint
  - Publish to the platform endpoint
  - Works with Google GCM, Apple APNS, Amazon ADM…

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Amazon SNS – Security

- **Encryption:**
  - In-flight encryption using HTTPS API
  - At-rest encryption using KMS keys
  - Client-side encryption if the client wants to perform encryption/decryption itself

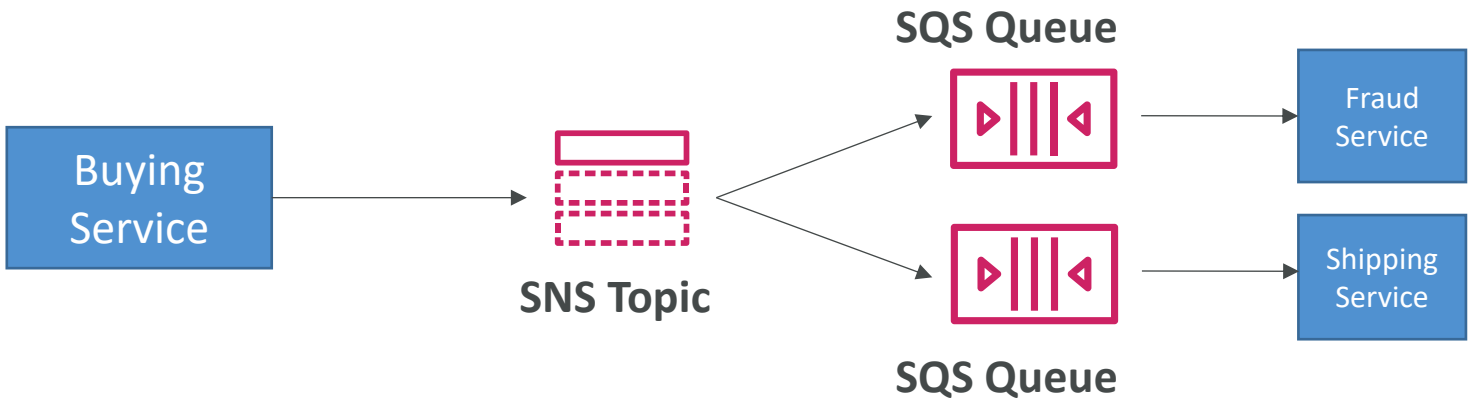
- **Access Controls:** IAM policies to regulate access to the SNS API

- **SNS Access Policies** (similar to S3 bucket policies)
  - Useful for cross-account access to SNS topics
  - Useful for allowing other services (S3…) to write to an SNS topic

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

SNS + SQS: Fan Out



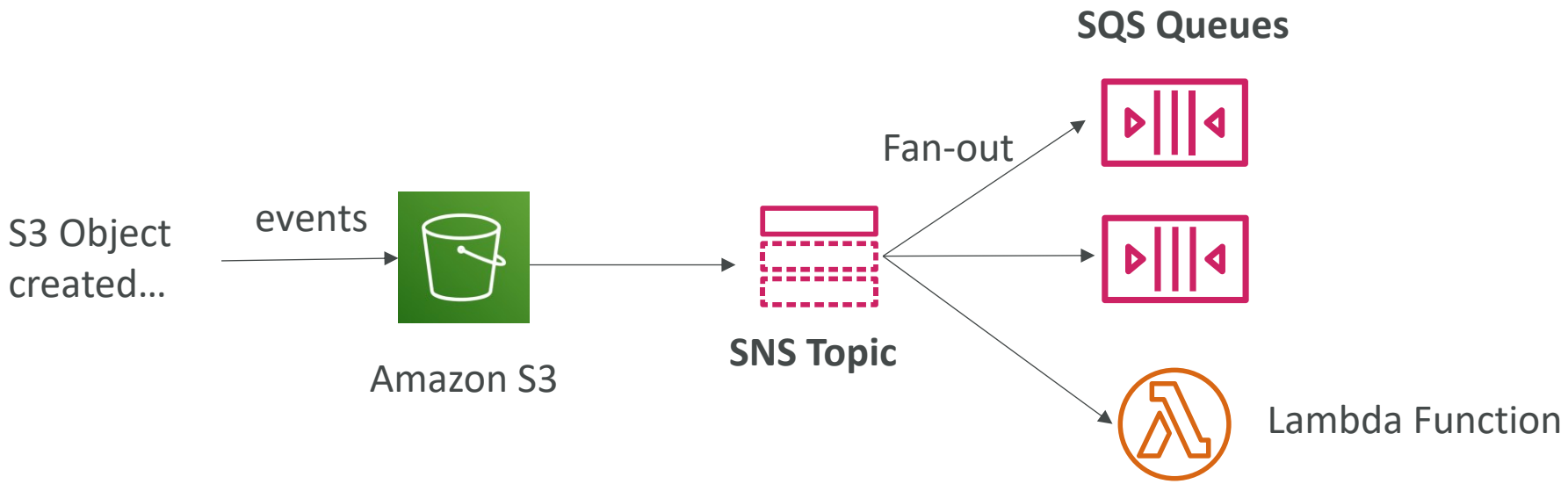
- Push once in SNS, receive in all SQS queues that are subscribers
- Fully decoupled, no data loss
- SQS allows for: data persistence, delayed processing and retries of work
- Ability to add more SQS subscribers over time
- Make sure your SQS queue **access policy** allows for SNS to write
- Cross-Region Delivery: works with SQS Queues in other regions

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Application: S3 Events to multiple queues

- For the same combination of: **event type** (e.g. object create) and **prefix** (e.g. images/) you can only have <u>one S3 Event rule</u>
- If you want to send the same S3 event to many SQS queues, use fan-out



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Application: SNS to Amazon S3 through Kinesis Data Firehose

- SNS can send to Kinesis and therefore we can have the following solutions architecture:

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Amazon SNS – FIFO Topic

- FIFO = First In First Out (ordering of messages in the topic)

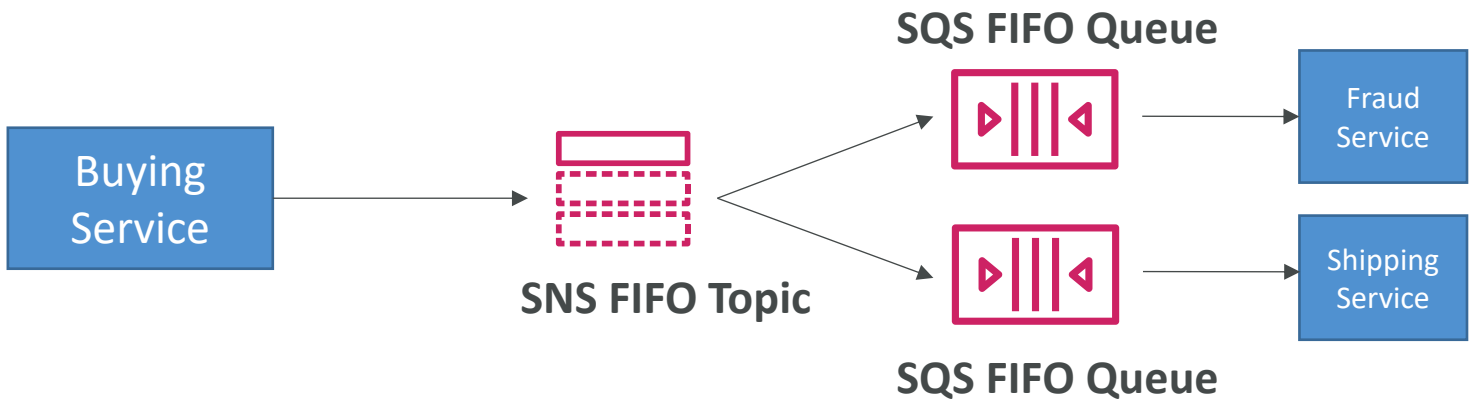
- Similar features as SQS FIFO:
  - **Ordering** by Message Group ID (all messages in the same group are ordered)
  - **Deduplication** using a Deduplication ID or Content Based Deduplication
- **Can have SQS Standard and FIFO queues as subscribers**
- Limited throughput (same throughput as SQS FIFO)

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

SNS FIFO + SQS FIFO: Fan Out

- In case you need fan out + ordering + deduplication

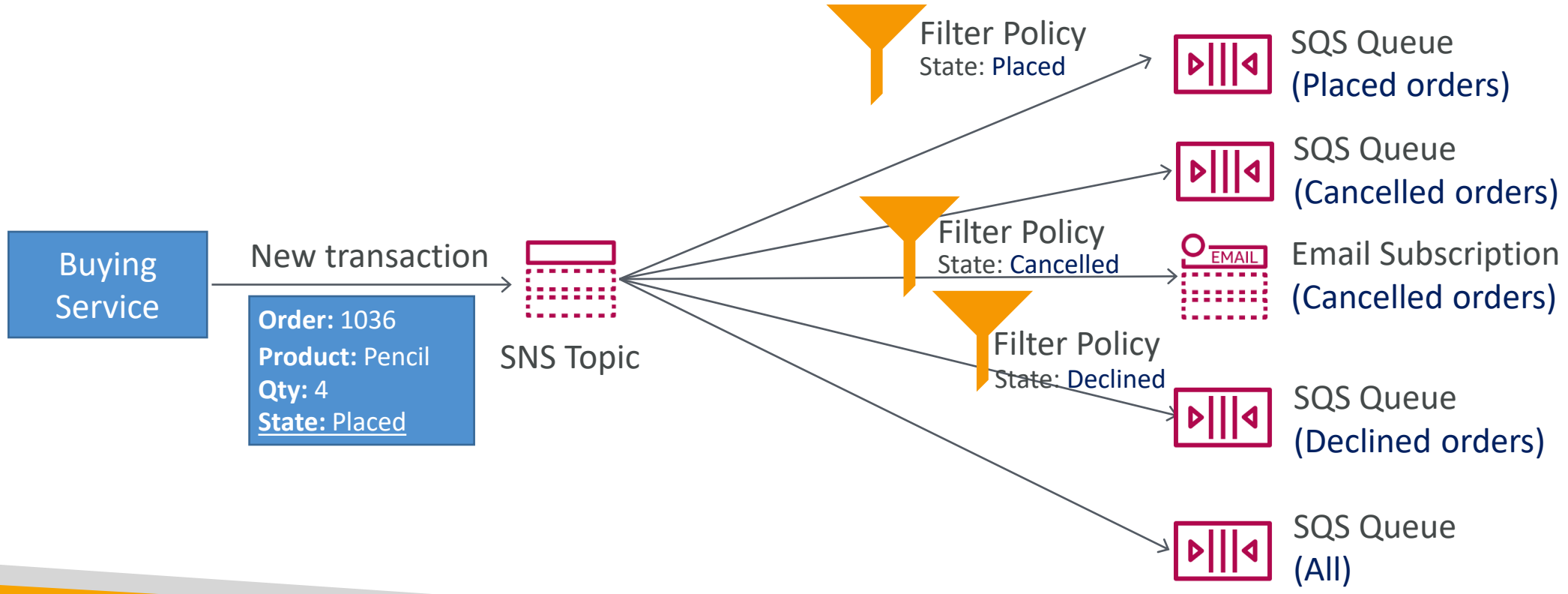


NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

SNS – Message Filtering

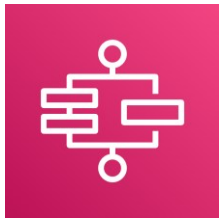
- JSON policy used to filter messages sent to SNS topic's subscriptions
- If a subscription doesn't have a filter policy, it receives every message



© 2023 All Rights Reserved Worldwide

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

AWS Step Functions



- **Use to design workflows**
- Easy visualizations
- Advanced Error Handling and Retry mechanism outside the code
- Audit of the history of workflows
- Ability to “Wait” for an arbitrary amount of time
- Max execution time of a State Machine is 1 year

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

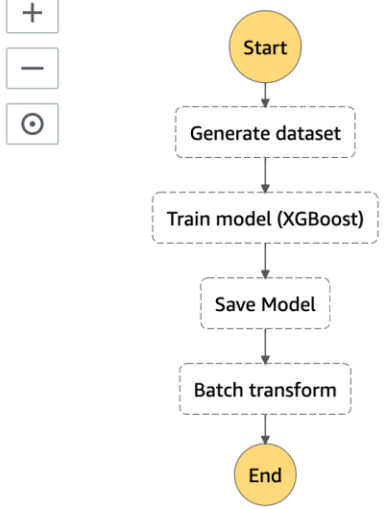
© 2023 All Rights Reserved Worldwide

Step Functions – Examples
Train a Machine Learning Model

Definition

Code is pre-configured by the chosen sample project. It can be edited after creation. Step Functions state machines are defined using the JSON-based Amazon States Language (ASL). Learn more

```
{
  "StartAt": "Generate dataset",
  "States": {
    "Generate dataset": {
      "Resource": "<GENERATE_LAMBDA_FUNCTION_ARN>",
      "Type": "Task",
      "Next": "Train model (XGBoost)"
    },
    "Train model (XGBoost)": {
      "Resource": "arn:<PARTITION>:states:::sagemaker:createTrainingJob.sync",
      "Parameters": {
        "AlgorithmSpecification": {
          "TrainingImage": "<SAGEMAKER_TRAINING_IMAGE>",
          "TrainingInputMode": "File"
        },
        "OutputDataConfig": {
          "S3OutputPath": "s3://<S3_BUCKET>/models"
        },
        "StoppingCondition": {
          "MaxRuntimeInSeconds": 86400
        },
        "ResourceConfig": {
          "InstanceCount": 1,
          "InstanceType": "ml.m4.xlarge",
```



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

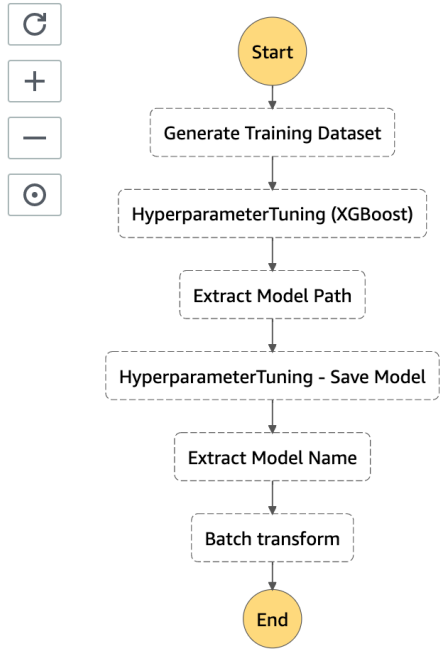
© 2023 All Rights Reserved Worldwide

Step Functions – Examples
Tune a Machine Learning Model

Definition

Code is pre-configured by the chosen sample project. It can be edited after creation. Step Functions state machines are defined using the JSON-based Amazon States Language (ASL). Learn more

```
{
    "StartAt": "Generate Training Dataset",
    "States": {
        "Generate Training Dataset": {
            "Resource": "<GENERATE_LAMBDA_FUNCTION_ARN>",
            "Type": "Task",
            "Next": "HyperparameterTuning (XGBoost)"
        },
        "HyperparameterTuning (XGBoost)": {
            "Resource": "arn:<PARTITION>:states:::sagemaker:createHyperParameterTuningJob.sync",
            "Parameters": {
                "HyperParameterTuningJobName.$": "<JOB_NAME_FROM_LAMBDA>",
                "HyperParameterTuningJobConfig": {
                    "Strategy": "Bayesian",
                    "HyperParameterTuningJobObjective": {
                        "Type": "Minimize",
                        "MetricName": "validation:rmse"
                    },
                    "ResourceLimits": {
                        "MaxNumberOfTrainingJobs": 2,
                        "MaxParallelTrainingJobs": 2
                    },
                    "ParameterRanges": {
```



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

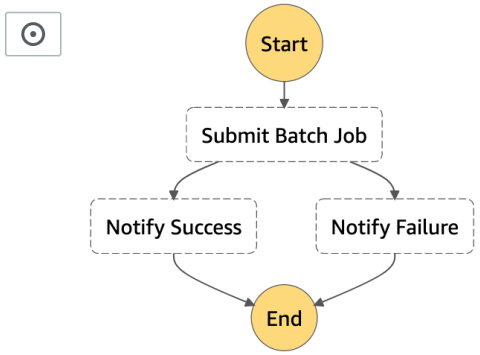
© 2023 All Rights Reserved Worldwide

Step Functions – Examples
Manage a Batch Job

Definition

Code is pre-configured by the chosen sample project. It can be edited after creation. Step Functions state machines are defined using the JSON-based Amazon States Language (ASL). Learn more

```
1  {
2    "Comment": "An example of the Amazon States Language for notification on an AWS Batch job completion",
3    "StartAt": "Submit Batch Job",
4    "TimeoutSeconds": 3600,
5    "States": {
6      "Submit Batch Job": {
7        "Type": "Task",
8        "Resource": "arn:<PARTITION>:states:::batch:submitJob.sync",
9        "Parameters": {
10         "JobName": "BatchJobNotification",
11         "JobQueue": "<BATCH_QUEUE_ARN>",
12         "JobDefinition": "<BATCH_JOB_DEFINITION_ARN>"
13       },
14       "Next": "Notify Success",
15       "Catch": [
16         {
17           "ErrorEquals": [ "States.ALL" ],
18           "Next": "Notify Failure"
19         }
20       ]
21     },
22     "Notify Success": {
23       "Type": "Task",
24       "Resource": "arn:<PARTITION>:states:::sns:publish",
```

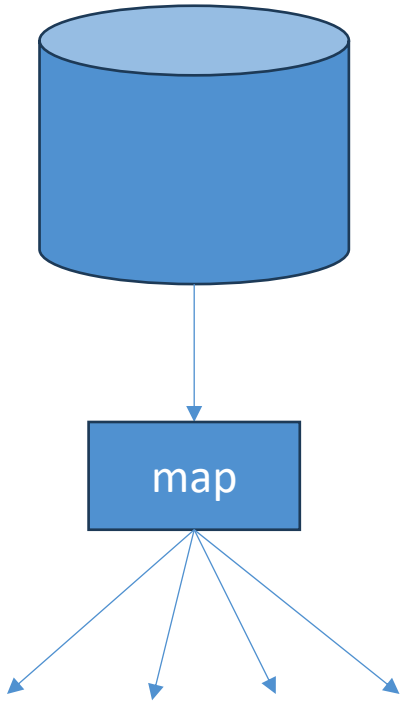


NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

AWS Step Functions

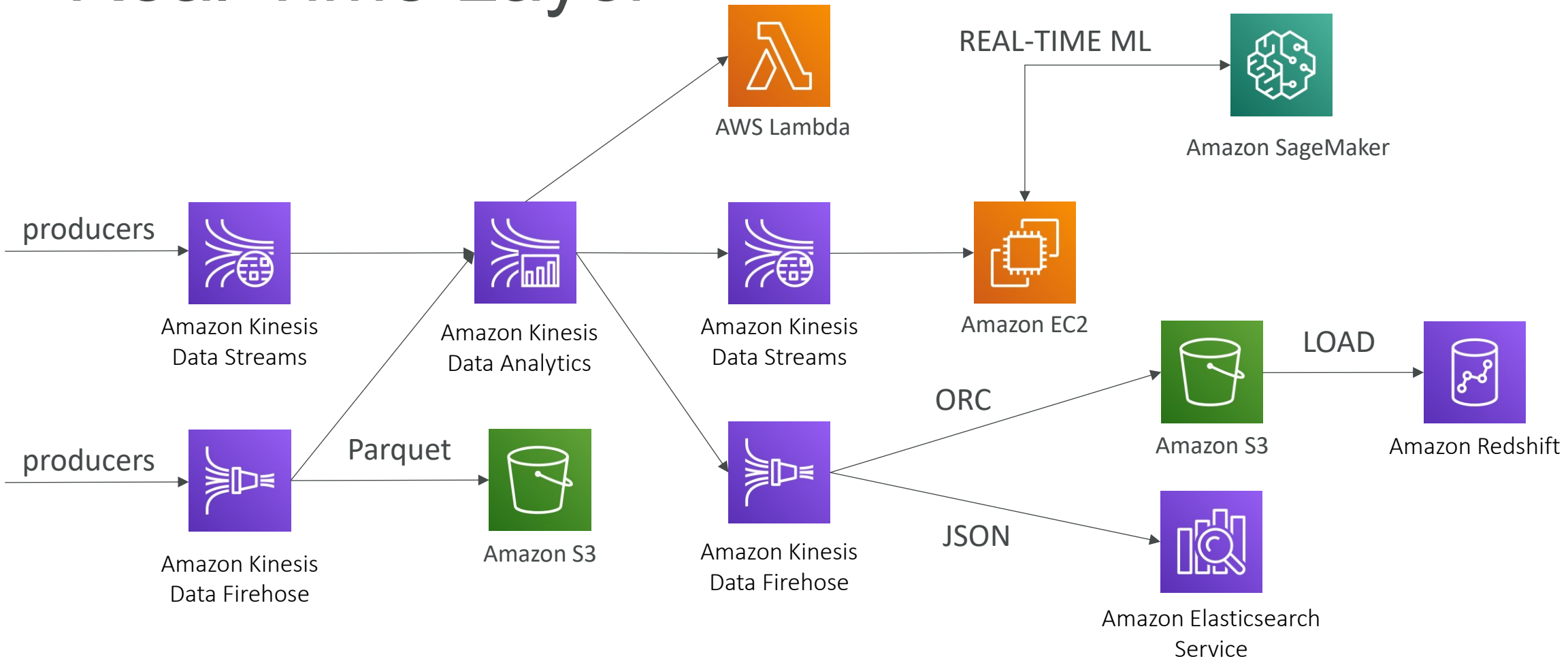
- Your workflow is called a *state machine*
- Each step in a workflow is a *state*
- Types of states
  - **Task**: Does something with Lambda, other AWS services, or third party API's
  - **Choice**: Adds conditional logic via Choice Rules (ie, comparisons)
  - **Wait**: Delays state machine for a specified time
  - **Parallel**: Add separate branches of execution
  - **Map**: Run a set of steps for each item in a dataset, in parallel
    - **This is most relevant to data engineering! Works with JSON, S3 objects, CSV files**
  - Also **Pass**, **Succeed**, **Fail**



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

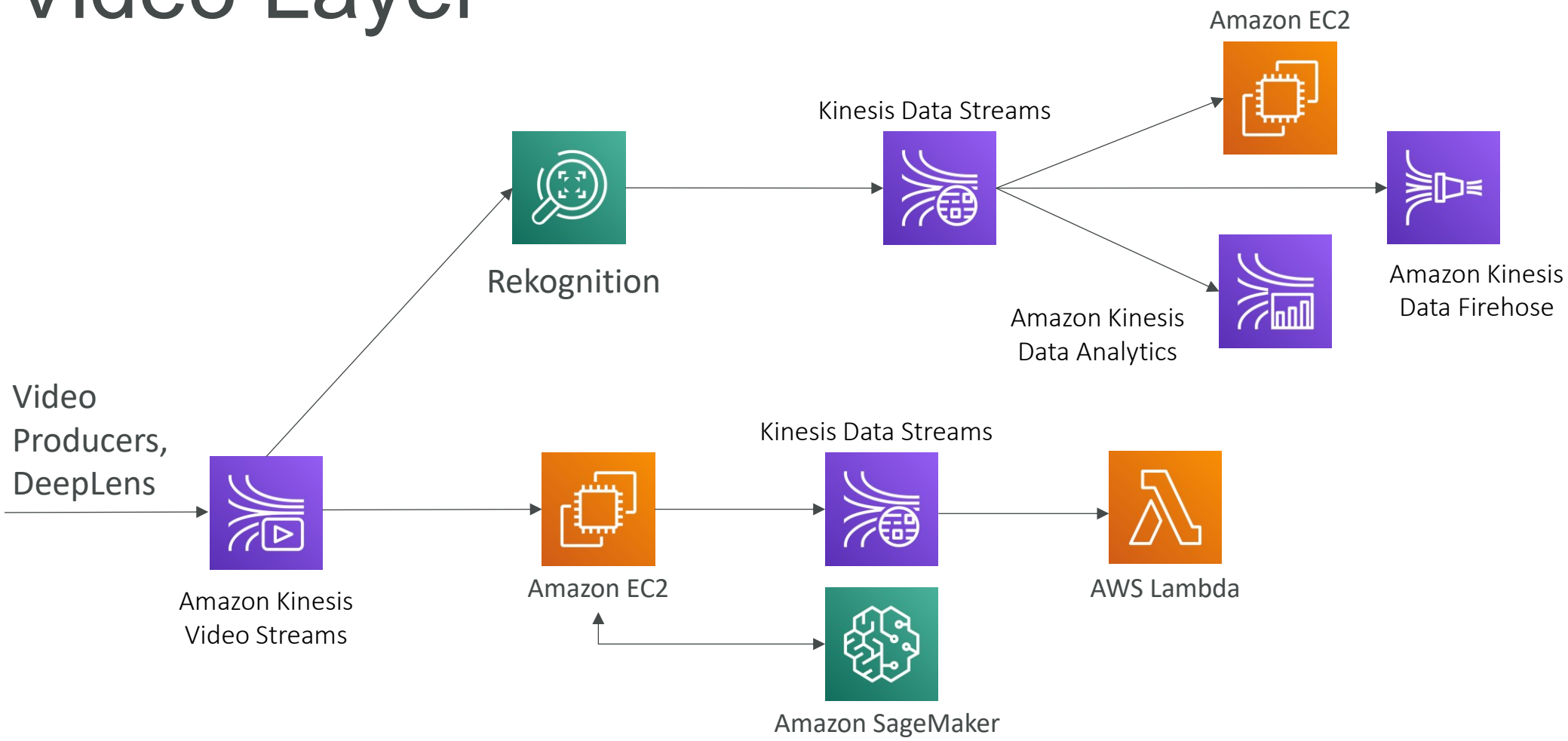
Full Data Engineering Pipeline Real-Time Layer



© 2023 All Rights Reserved Worldwide

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

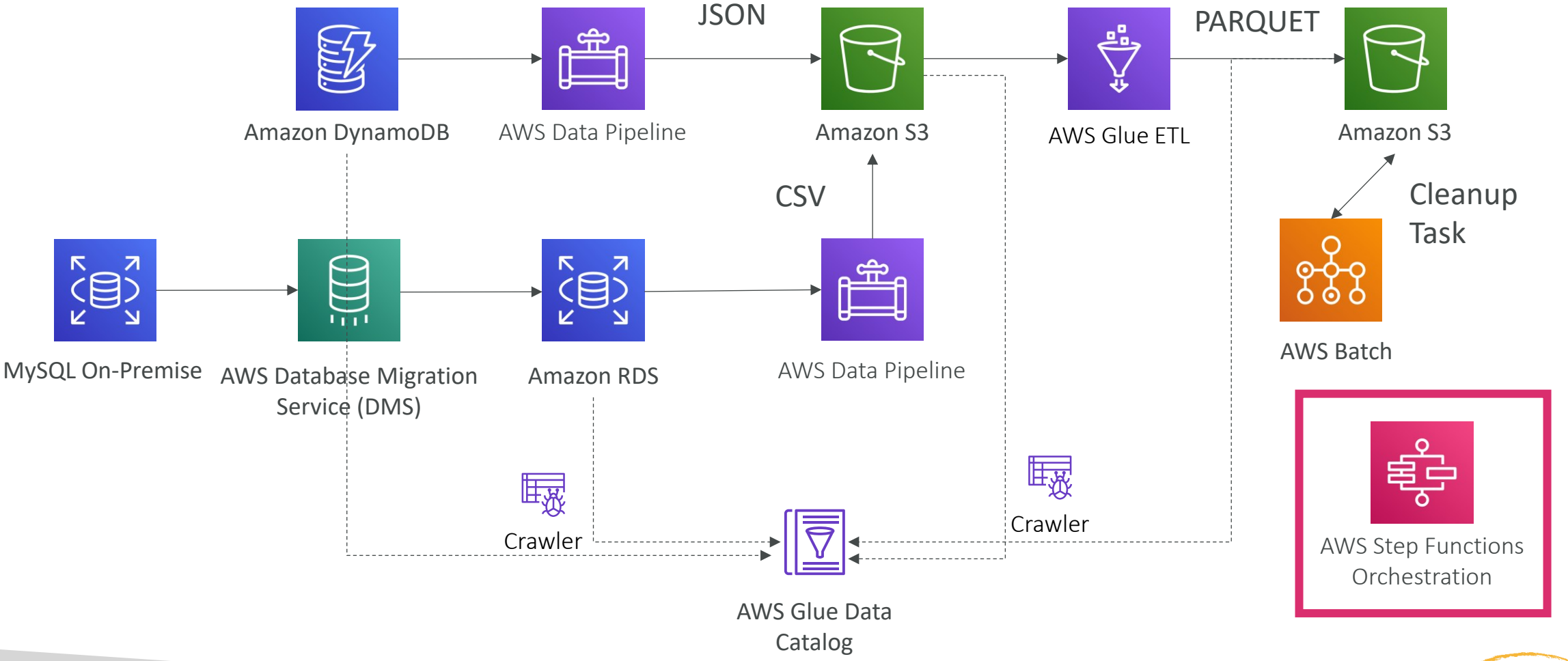
Full Data Engineering Pipeline Video Layer



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

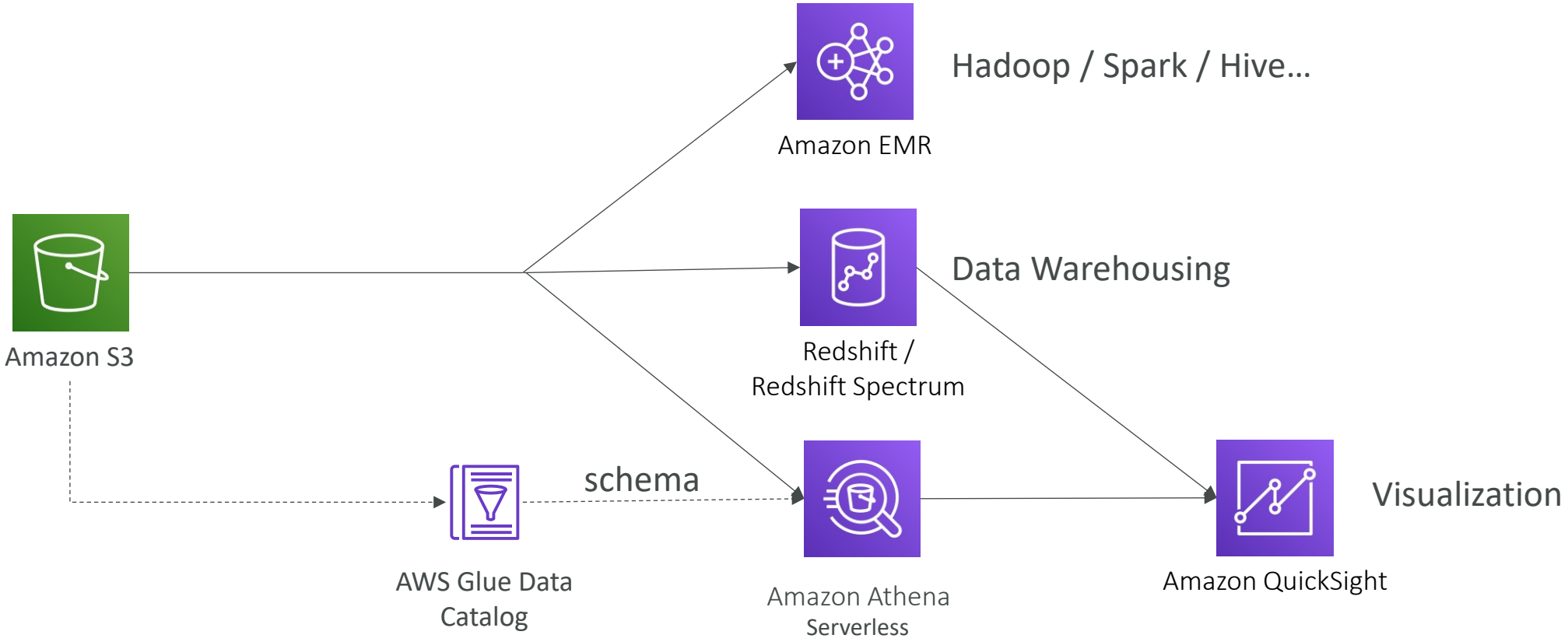
Full Data Engineering Pipeline Batch Layer



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Full Data Engineering Pipeline
Analytics layer



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Amazon AppFlow

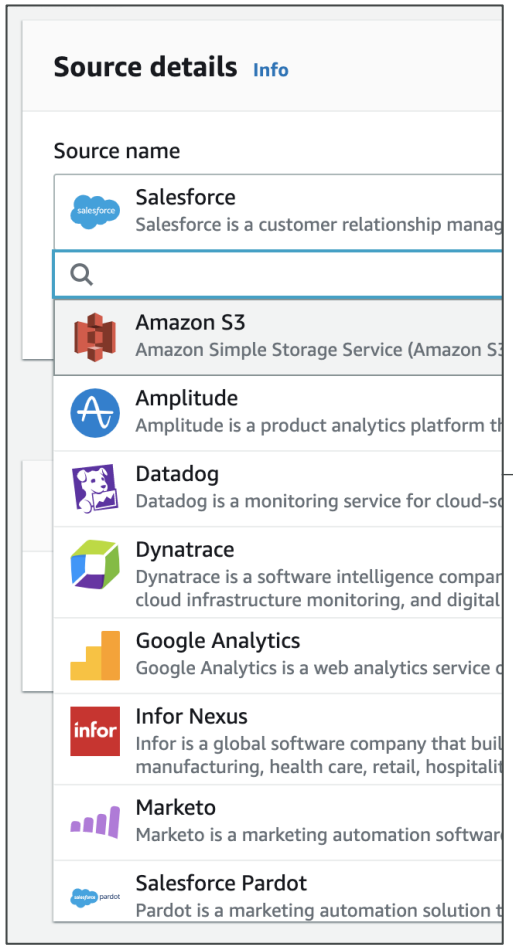
- Fully managed integration service that enables you to securely transfer data between **Software-as-a-Service (SaaS) applications and AWS**
- Sources: **Salesforce**, SAP, Zendesk, Slack, and ServiceNow
- Destinations: AWS services like **Amazon S3, Amazon Redshift** or non-AWS such as SnowFlake and Salesforce
- Frequency: on a schedule, in response to events, or on demand
- Data transformation capabilities like filtering and validation
- Encrypted over the public internet or privately over AWS PrivateLink
- Don't spend time writing the integrations and leverage APIs immediately

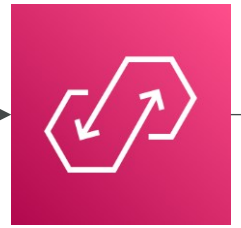
NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Amazon AppFlow

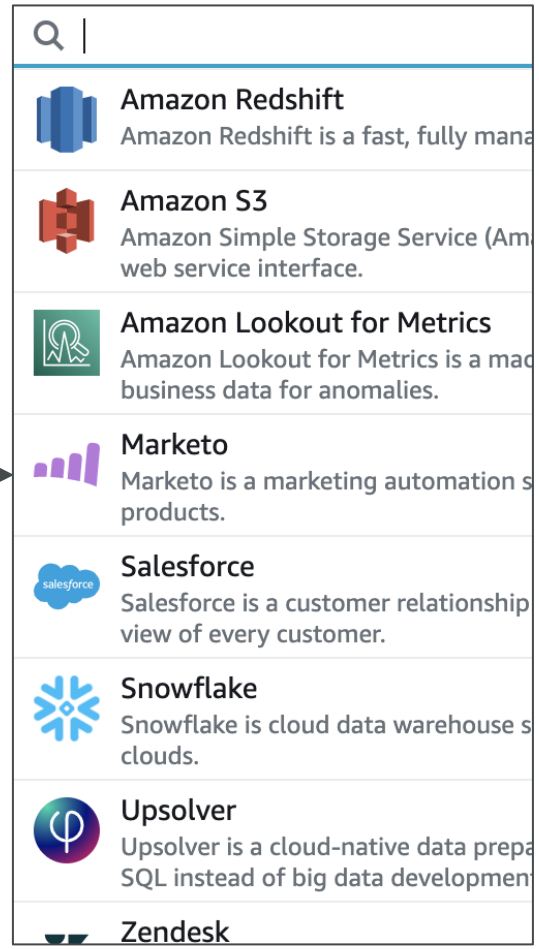
Sources





Amazon AppFlow

Destinations

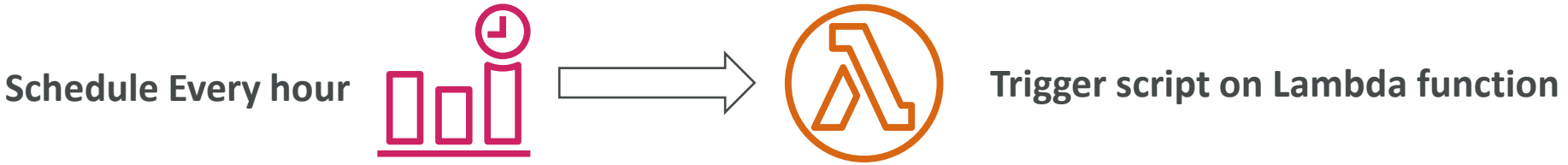


NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

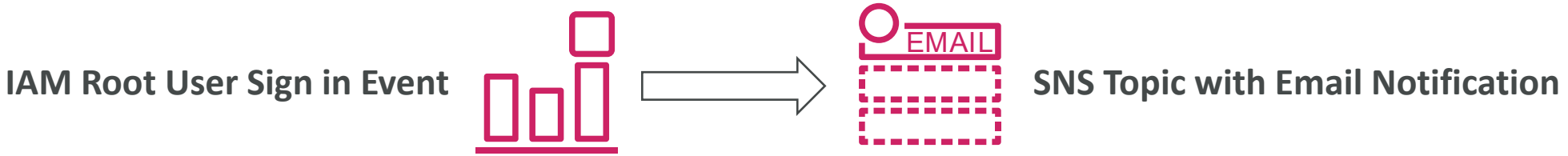
© 2023 All Rights Reserved Worldwide

Amazon EventBridge (formerly CloudWatch Events)

- Schedule: Cron jobs (scheduled scripts)



- Event Pattern: Event rules to react to a service doing something

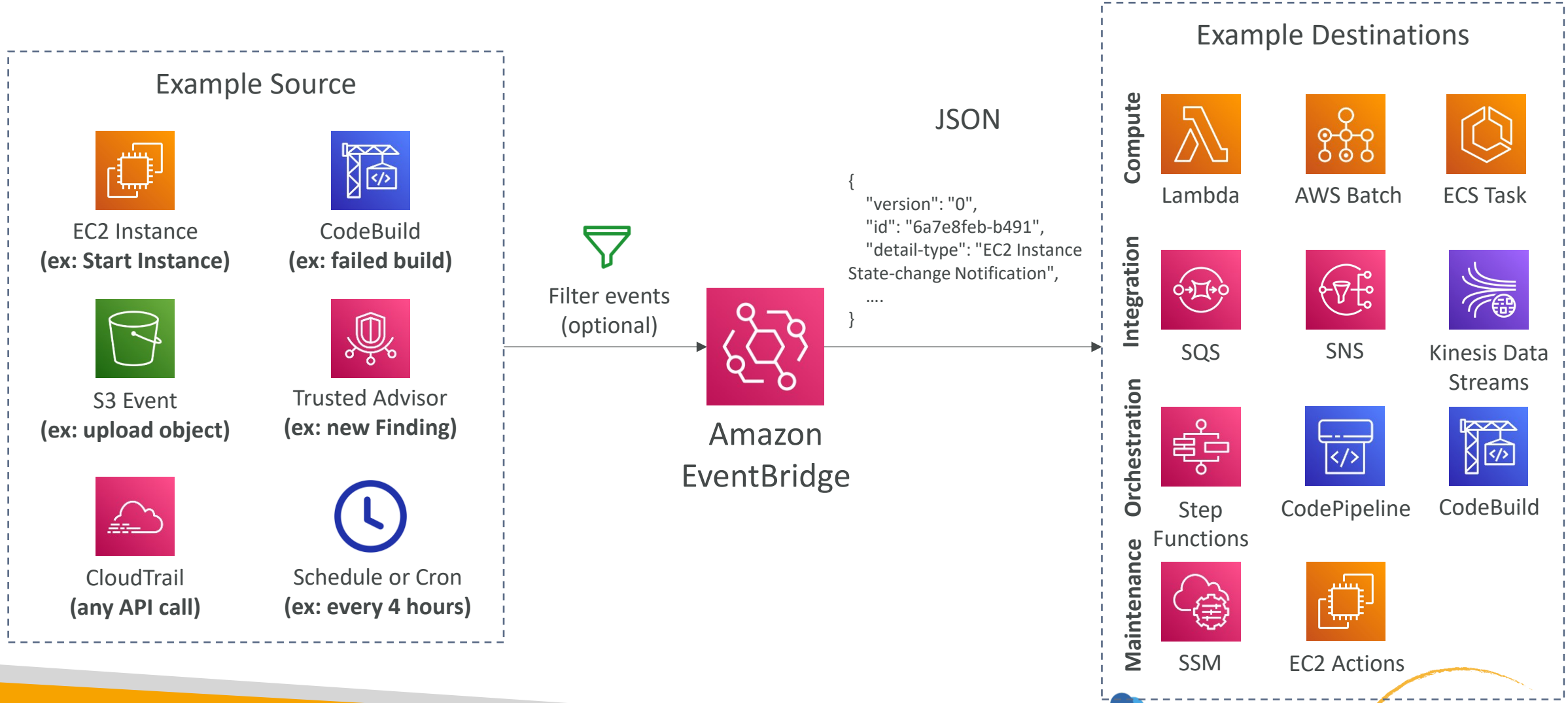


- Trigger Lambda functions, send SQS/SNS messages…

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

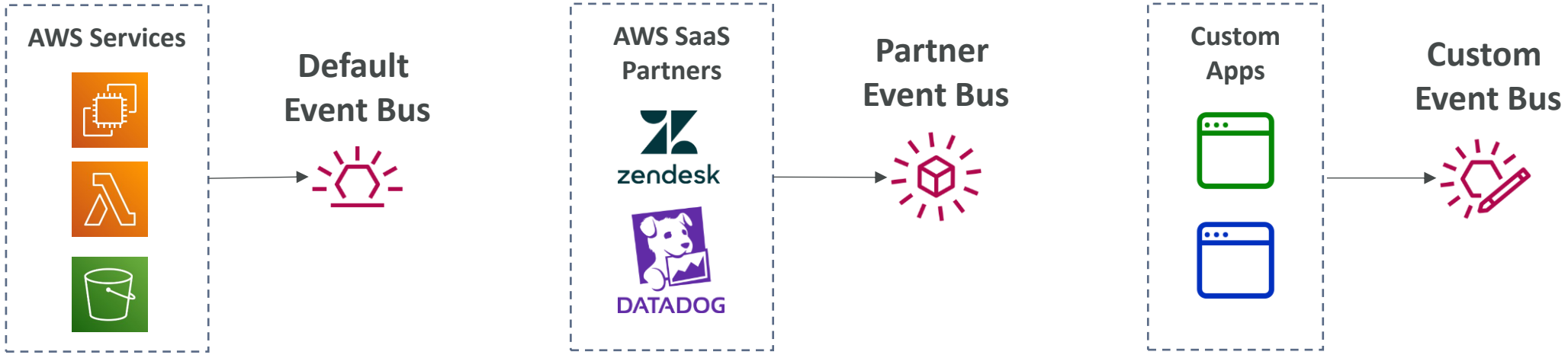
Amazon EventBridge Rules



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Amazon EventBridge



- Event buses can be accessed by other AWS accounts using Resource-based Policies

- You can **archive events** (all/filter) sent to an event bus (indefinitely or set period)
- Ability to **replay archived events**

© 2023 All Rights Reserved Worldwide

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon EventBridge – Schema Registry

- EventBridge can analyze the events in your bus and infer the **schema**
- The **Schema Registry** allows you to generate code for your application, that will know in advance how data is structured in the event bus
- Schema can be versioned

aws.codepipeline@CodePipelineActionExecuti…

Schema details

Schema name	Last modified	Schema ARN
aws.codepipeline@CodePipelineActionExecutionStateChange	Dec 1, 2019, 12:11 AM GMT	-

Schema registry	Number of versions	Schema type
aws.events	1	OpenAPI 3.0

Description

Schema for event type CodePipelineActionExecutionStateChange, published by AWS service aws.codepipeline

Version 1 Created on Dec 1, 2019, 12:11 AM GMT

Action ▼ Download code bindings

```
1 {
2   "openapi": "3.0.0",
3   "info": {
4     "version": "1.0.0",
5     "title": "CodePipelineActionExecutionStateChange"
6   },
7   "paths": {},
8   "components": {
9     "schemas": {
10      "AWSEvent": {
```

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

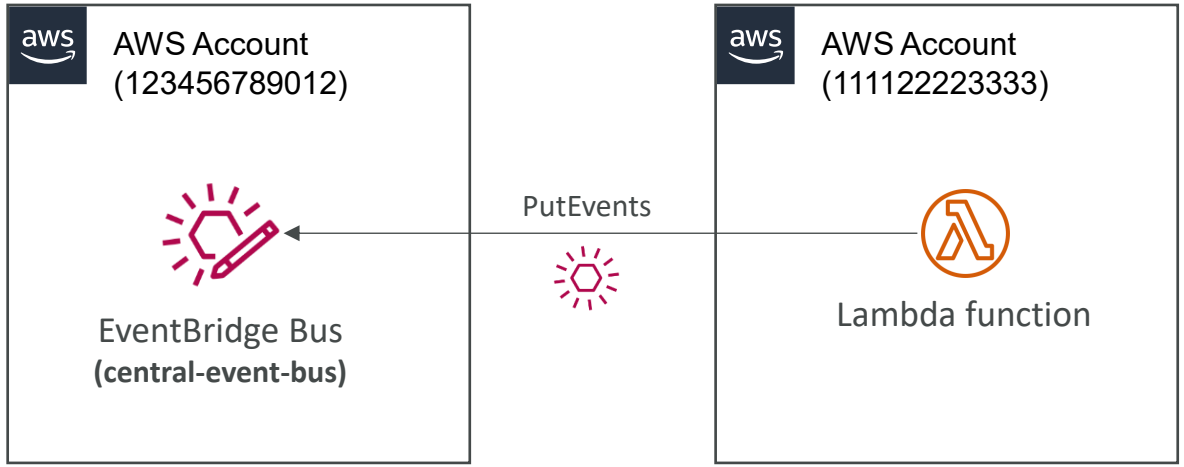
© 2023 All Rights Reserved Worldwide

Amazon EventBridge – Resource-based Policy

- Manage permissions for a specific Event Bus
- Example: allow/deny events from another AWS account or AWS region
- Use case: aggregate all events from your AWS Organization in a single AWS account or AWS region

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": "events:PutEvents",
            "Principal": { "AWS": "111122223333" },
            "Resource": "arn:aws:events:us-east-1:123456789012:
event-bus/central-event-bus"
        }
    ]
}
```

Allow **events** from another AWS account



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Amazon Managed Workflows for Apache Airflow (MWAA)

- Apache Airflow is batch-oriented workflow tool
- Develop, schedule, and monitor your workflows
- Workflows are defined as Python code that creates a Directed Acyclic Graph (DAG)
- Amazon MWAA provides a managed service for Apache Airflow so you don't have to deal with installing or maintaining it
- Use cases:
  - Complex workflows
  - ETL coordination
  - Preparing ML data

Apache Airflow

Amazon Managed Workflows for Apache Airflow

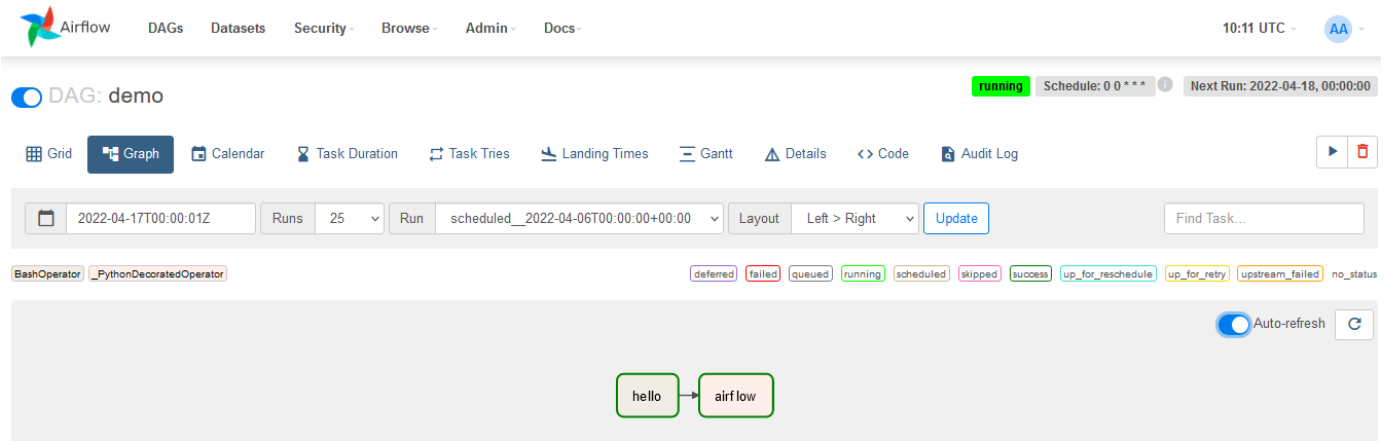
NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

DataCumulus Sundog Education

Sample DAG in Airflow

```
from datetime import datetime

from airflow import DAG
from airflow.decorators import task
from airflow.operators.bash import BashOperator

# A DAG represents a workflow, a collection of tasks
with DAG(dag_id="demo", start_date=datetime(2022, 1, 1),
schedule="0 0 * * *") as dag:

    # Tasks are represented as operators
    hello = BashOperator(task_id="hello", bash_command="echo
hello")

    @task()
    def airflow():
        print("airflow")

    # Set dependencies between tasks
    hello >> airflow()
```

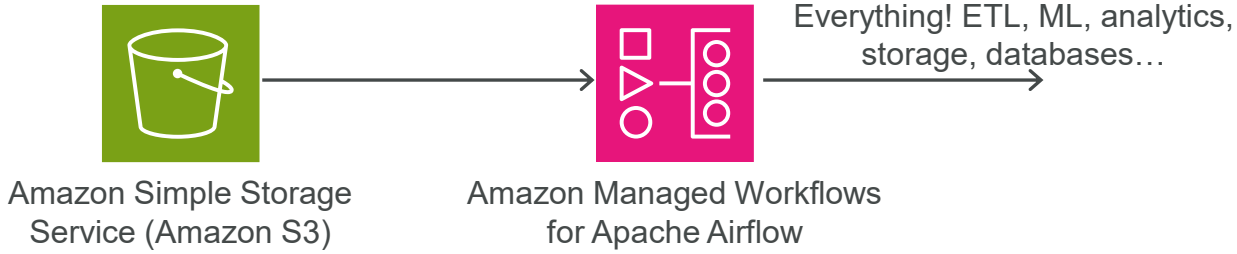


NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Airflow + MWAA

- Your DAGs (Python code) are uploaded into S3
  - May also zip it together with required plugins and requirements
- Amazon MWAA picks it up, and orchestrates and schedules the pipelines defined by each DAG.
- Runs within a VPC
  - In at least 2 AZ's
- Private or public endpoints
  - IAM-managed
  - (Access to Airflow Web Server)
- Automatic scaling
  - Airflow Workers autoscale up to the limits you define



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

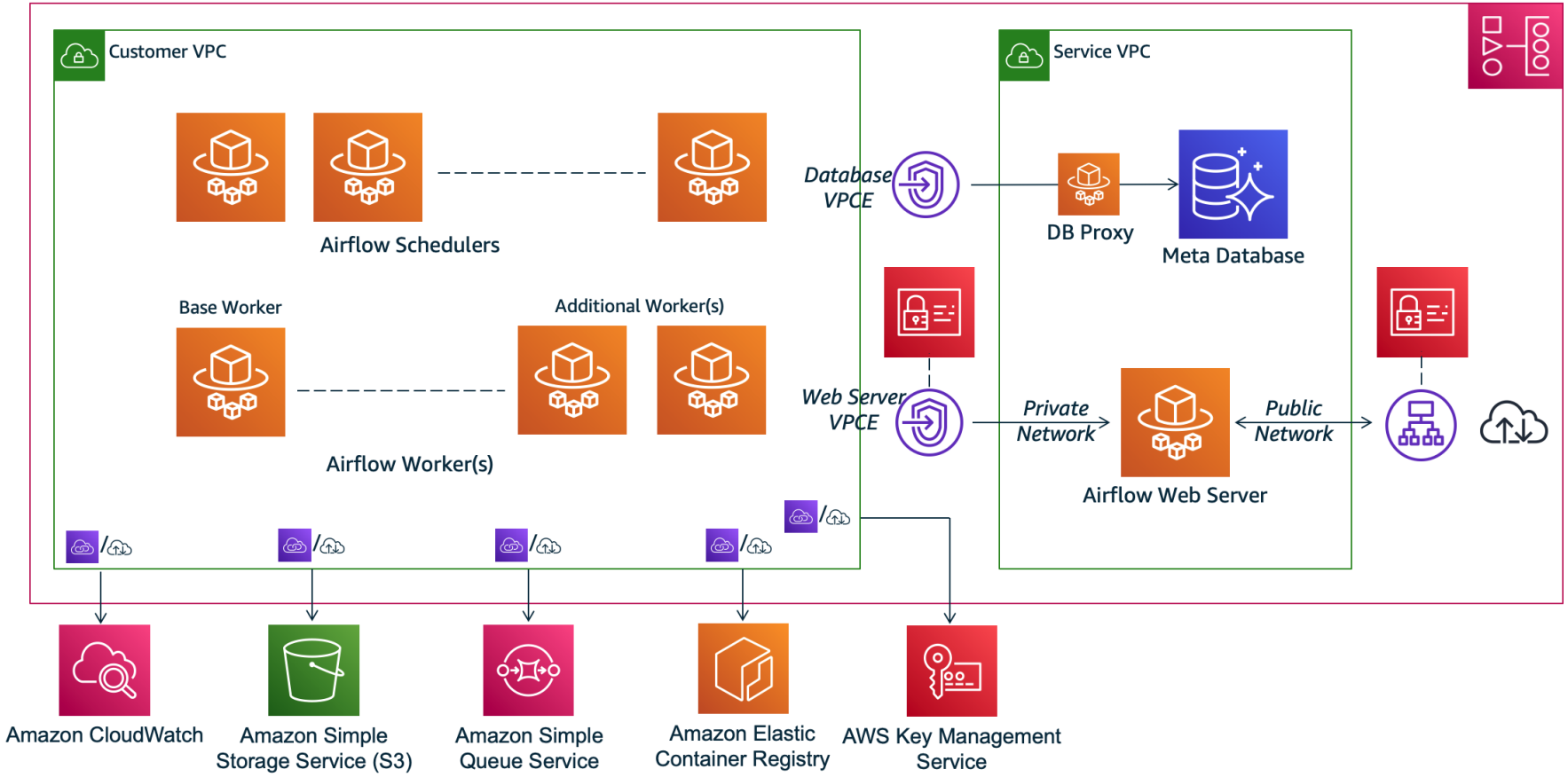
Amazon MWAA Integration

- Leverages open-source integrations
  - Athena, Batch, CloudWatch, DynamoDB, DataSync
  - EMR, Fargate, EKS, Kinesis, Glue, Lambda
  - Redshift, SQS, SNS, SageMaker, S3… and more
  - Security services (AWS Secrets Manager, etc)
- Schedulers and Workers are AWS Fargate containers

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide

Amazon MWAA Architecture



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© 2023 All Rights Reserved Worldwide